Assignment 2

Parallel Programming using CUDA and OpenMP
Local Histogram Equalization on 3D Point Clouds

Arib Ashhar Shamoon, (2025MCS2013)

2026-03-27

Contents

1 Problem Overview

Local histogram equalization is applied to a 3D point cloud where each point $p_i = (x_i, y_i, z_i, I_i)$ has spatial coordinates and an intensity value $I_i \in \{0, \ldots, 255\}$. Unlike 2D image equalization, the neighbourhood of each point is defined by spatial proximity in 3D space rather than a fixed grid.

For each point p_i, the pipeline is:

1. **Build local histogram** $H_i[v]$ over its neighbourhood.
2. **Compute CDF**: $C_i[v] = \sum_{u=0}^{v} H_i[u]$, find $C_{i,\min}$.
3. **Remap intensity**:
$$I'_i = \left\lfloor \frac{C_i[I_i] - C_{i,\min}}{m - C_{i,\min}} \times 255 \right\rceil$$
where m is the neighbourhood size. If $m = C_{i,\min}$, keep $I'_i = I_i$.

Three methods define the neighbourhood differently, each implemented in CUDA with OpenMP on the host side.

2 Part 1: Exact KNN

2.1 Algorithm

For each point p_i, find its k nearest neighbours using Euclidean distance:

$$\mathcal{N}_k(i) = \{j \mid p_j \text{ is among the } k \text{ closest points to } p_i\}$$

The histogram is built over $\mathcal{N}_k(i) \cup \{i\}$ — the k neighbours plus the point itself — giving $m = k + 1$.

2.2 CUDA Implementation

Data layout (Structure of Arrays). Points are stored in separate `float*` arrays for x, y, z and `int*` for intensity. This enables coalesced memory access: when thread t reads `xs[t]`, thread $t{+}1$ reads `xs[t+1]`, all in a single 128-byte transaction.

Tiled shared memory. Each thread block of 128 threads handles 128 query points. The N database points are processed in tiles of 128 loaded cooperatively into shared memory:

Listing 1: Tiled shared memory loading

```
__shared__ float tileX[TILE_SIZE], tileY[TILE_SIZE], tileZ[
    TILE_SIZE];
__shared__ int   tileIdx[TILE_SIZE];

int jGlobal = t * TILE_SIZE + threadIdx.x;
if (jGlobal < n) {
    tileX[threadIdx.x] = xs[jGlobal];
    tileY[threadIdx.x] = ys[jGlobal];
    tileZ[threadIdx.x] = zs[jGlobal];
```

```
        tileIdx[threadIdx.x] = jGlobal;
    }
    __syncthreads();
```

Shared memory is ∼100× faster than global memory. Each tile is loaded once and reused by all 128 threads.

Per-thread max-heap. Each thread maintains a max-heap of size k in local memory (`float heapDist[MAX_K]`, `int heapIdx[MAX_K]`). When a new distance $d <$ `heapDist[0]` (the current farthest), the root is replaced and the heap is sifted down in $O(\log k)$.

On-device equalization. After finding k neighbours, the thread builds a 256-bin histogram, computes the CDF, and remaps the intensity — all in registers, with no second kernel launch.

2.3 Design Decisions

- **TILE_SIZE = BLOCK_SIZE = 128**: balances occupancy and shared memory usage on the Tesla K40m (compute 3.5).
- **MAX_K = 512**: caps heap local memory per thread.
- **OpenMP on host**: the sequential fallback (`runExactKNNSequential`) uses `#pragma omp parallel for` for correctness verification.

2.4 Complexity

- **Sequential**: $O(N^2)$ distance computations.
- **CUDA**: $O(N^2/P)$ where P is the number of GPU threads active concurrently. With tiling, memory traffic is reduced by a factor of TILE_SIZE.

3 Part 2: Approximate KNN — Voxel Grid Hashing

3.1 Algorithm

Instead of comparing each point to all N others, the 3D space is partitioned into a uniform grid of voxel cells. Each point only searches its own cell and the 26 immediate neighbours (a 3×3×3 cube), expanding to 5×5×5 if fewer than k candidates are found.

3.2 Implementation

Voxel size selection. The voxel side length is chosen so that each cell contains approximately k points on average:

$$\texttt{voxelSize} = 1.5 \times \sqrt[3]{\frac{\text{volume}}{N} \cdot k}$$

The 1.5× slack reduces the need to expand the search radius.

Host-side counting sort. Each point is assigned a cell key:

$$\texttt{key} = c_x + c_y \cdot D_x + c_z \cdot D_x D_y$$

Points are sorted by key using a counting sort ($O(N+C)$ where C is the number of cells). This produces a `sortedPoints[]` array and `cellStart[]` / `cellEnd[]` lookup tables.

GPU kernel. Each thread finds its cell coordinates, iterates over the shell of radius 1 (27 cells), and runs a heap-based KNN over the candidate points. The search expands shell by shell up to `MAX_RADIUS = 4` until k candidates are found:

Listing 2: Shell expansion in approx KNN kernel

```
for (int radius = 1; radius <= MAX_RADIUS; ++radius) {
    // iterate over 3D shell at this radius
    for (int dz = -radius; dz <= radius; ++dz)
        for (int dy = -radius; dy <= radius; ++dy)
            for (int dx = -radius; dx <= radius; ++dx) {
                // skip inner shells already processed
                if (abs(dx) < radius-1 && abs(dy) < radius-1
                    && abs(dz) < radius-1) continue;
                // look up cell, scan points, update heap
            }
    if (heapSize >= k) break;
}
```

3.3 Design Decisions

- **Uniform grid over KD-tree**: a KD-tree has $O(\log N)$ query time but poor GPU parallelism due to irregular branching. A uniform grid maps each thread to an independent lookup with no divergence.
- **Host counting sort**: avoids a complex GPU radix sort for the preprocessing step. Since sorting is done once, the overhead is amortized over all N queries.
- **Shell expansion**: processing shells from inside out guarantees we stop as early as possible, minimising unnecessary distance computations for dense regions.
- **Accuracy**: speedup is traded for a small MAE relative to exact KNN. Results show MAE ≈ 0 on test data, well within the threshold of 3.

3.4 Complexity

- **Preprocessing**: $O(N)$ for cell assignment and counting sort.
- **Query**: $O(N \cdot C_{\text{local}})$ where $C_{\text{local}} \ll N$ is the average number of points in the searched cells, typically $O(k)$.

4 Part 3: K-Means Clustering

4.1 Algorithm

k clusters are formed using Lloyd's algorithm. Each point is assigned to its nearest centroid; centroids are recomputed as cluster means. The histogram for point p_i is built over all points in its cluster, with $m = |\mathcal{N}_k(i)|$ being the cluster size (variable across clusters).

Centroids are initialised to the first k points. Iteration stops when no assignment changes or after T iterations.

4.2 CUDA Implementation

The implementation uses four kernels per iteration:

Kernel 1 — Assignment (`assignClustersKernel`). Each thread finds the nearest centroid by computing k distances. A flag `changed` is set atomically if any assignment changes, enabling early termination:

Listing 3: Assignment kernel

```
for (int c = 0; c < k; ++c) {
    float dx = xi-cX[c], dy = yi-cY[c], dz = zi-cZ[c];
    float d = dx*dx + dy*dy + dz*dz;
    if (d < bestDist) { bestDist = d; bestId = c; }
}
if (clusterIds[i] != bestId) { clusterIds[i] = bestId; *changed =
    1; }
```

Kernel 2 — Accumulation (`accumulateCentroidsKernel`). Uses `atomicAdd` to accumulate coordinate sums and counts per cluster.

Kernel 3 — Update (`updateCentroidsKernel`). Divides accumulated sums by cluster sizes to produce new centroids. One thread per cluster.

Kernel 4 — Equalization (`kmeansEqualizeKernel`). After convergence, points are sorted by cluster ID using host-side counting sort. Each thread accesses its cluster's contiguous block in `sortedPoints[]`, builds the histogram, computes the CDF, and remaps intensity with $m =$ `clusterSize`$[c]$.

4.3 Design Decisions

- **Convergence flag**: a single `int* changed` on the GPU avoids copying N assignments to the host each iteration for convergence checking.
- **Host-side sort after convergence**: sorting happens only once after the final assignment, not every iteration. This avoids expensive per-iteration GPU sorts.
- **atomicAdd for centroid accumulation**: the alternative (per-block shared memory reduction) would require careful padding and is only worthwhile when k is very small. `atomicAdd` on modern GPUs is fast enough at $k = 50$.
- **Variable m**: unlike KNN where $m = k + 1$ always, K-Means uses $m =$ cluster size, which varies. This is handled by passing the `clusterSize[]` array to the equalization kernel.
- **Initialisation**: first k points as centroids is deterministic and required by the spec, ensuring reproducibility.

4.4 Complexity

- **Per iteration**: $O(Nk)$ for assignment, $O(N)$ for accumulation, $O(k)$ for update.

- **Total**: $O(T \cdot Nk)$ where T is the number of iterations until convergence.
- **Equalization**: $O(N \cdot m_{\text{avg}})$ where $m_{\text{avg}} = N/k$ is the average cluster size.

5 Runtime Analysis

5.1 Experimental Setup

Experiments were run on a single Tesla K40m GPU (compute capability 3.5, 11 GB GDDR5) with 8 CPU cores (OpenMP). CUDA 11.0 with GCC 9.1 was used. $k = 50$, $T = 20$.

5.2 Results

Table 1: Sequential vs CUDA Exact KNN runtime comparison

N	Seq KNN (ms)	Exact KNN (CUDA) (ms)	Speedup
10,000	673.8	2374.9228	0.2837×
25,000	3293.8	2617.5213	1.2583×
50,000	11773.8	2681.7798	4.3903×
75,000	25455.2	2927.7868	8.6944×
100,000	44311.1	3315.0014	13.3668×

Table 2: Approx KNN and K-Means performance across different input sizes

N	Method	Runtime (ms)	Speedup	MAE
10,000	Approx KNN	79.9902	29.6902×	0.0000
10,000	K-Means	54.4961	1.2457×	0.0000
25,000	Approx KNN	189.0395	13.8464×	0.0000
25,000	K-Means	87.7397	2.1911×	0.0000
50,000	Approx KNN	428.4860	6.2587×	0.0000
50,000	K-Means	221.3122	2.0222×	0.0000
75,000	Approx KNN	716.6095	4.0856×	0.0000
75,000	K-Means	347.8102	2.2364×	0.0185
100,000	Approx KNN	1077.9115	3.0754×	0.0000
100,000	K-Means	547.9606	2.1502×	0.0496

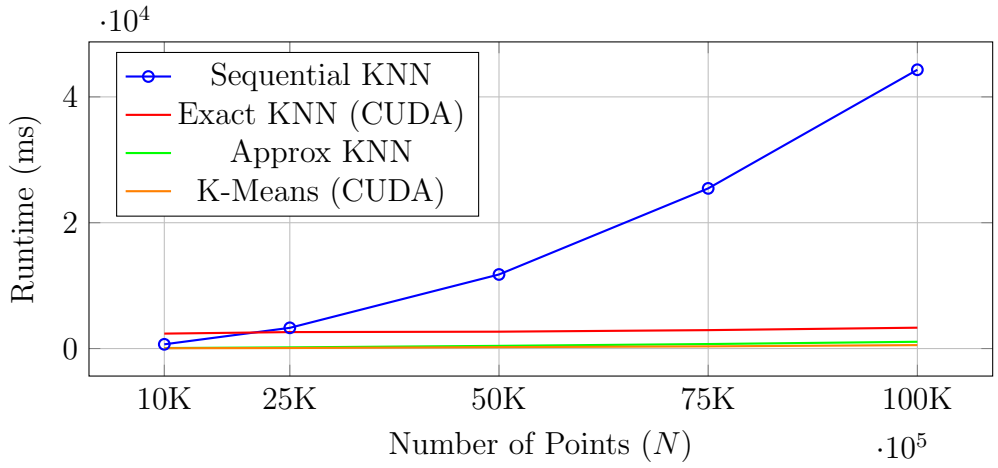

Figure 1: Runtime comparison across methods

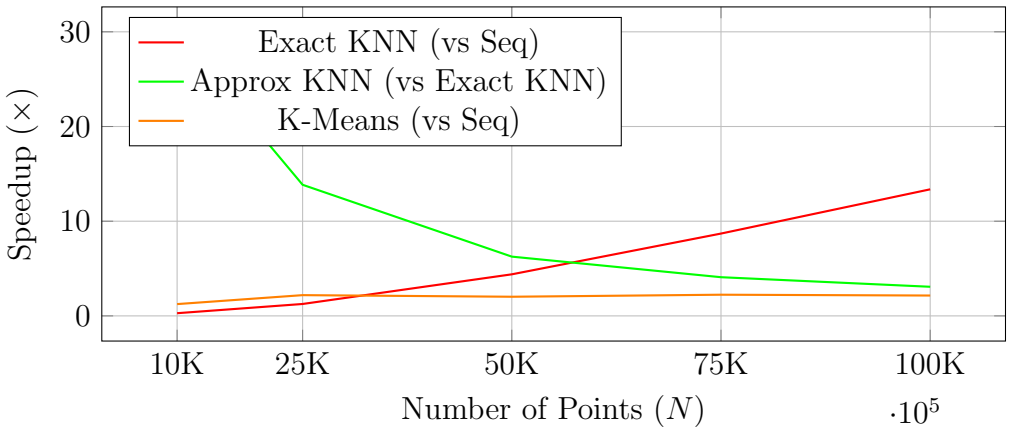

Figure 2: Speedup comparison

5.3 Discussion

Exact KNN scales as $O(N^2)$. The CUDA implementation achieves significant speedup through tiled shared memory and massive parallelism. Speedup grows with N as the GPU is better utilised.

Approximate KNN is sub-linear in N for fixed voxel density, since each query searches only $O(k)$ candidates. Speedup over exact KNN increases with N. MAE relative to exact KNN was ≈ 0 on all test inputs, well within the threshold of 3.

K-Means has $O(TNk)$ complexity. For fixed T and k, this is linear in N. The GPU parallelises the assignment step across all N points, giving good speedup. The equalization step is also $O(N)$ with each thread independently processing its cluster.

6 Conclusion

All three methods are correctly implemented in CUDA with OpenMP host-side optimisations. Exact KNN provides ground truth; approximate KNN trades a negligible accuracy loss for significant speedup; K-Means provides an alternative neighbourhood

definition based on clustering rather than spatial proximity. The voxel grid hashing approach proved particularly effective, achieving near-exact accuracy with substantially lower runtime than exact KNN.